FILE: C:\HCL_TRAINING\server\src\controllers\authController.js

LINES: 192 | SIZE: 5.0 KB | LANGUAGE: JS

MODULE: Recruiter Registration, Login, Session Verification, and Cookie Handlers

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');
const Organization = require('../models/Organization');
const ScoringConfig = require('../models/ScoringConfig');
const { logActivity } = require('../services/auditLogger');

const generateToken = (id) => {
  const secret = process.env.JWT_SECRET || 'hiresense_super_secure_jwt_secret_key_2026_recruit_intelligence';
  return jwt.sign({ id }, secret, {
    expiresIn: process.env.JWT_EXPIRES_IN || '7d',
  });
};

const sendTokenResponse = (user, statusCode, res) => {
  const token = generateToken(user._id);

  const cookieOptions = {
    expires: new Date(Date.now() + 7 * 24 * 60 * 60 * 1000), // 7 days
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'lax',
  };

  res.cookie('token', token, cookieOptions);

  res.status(statusCode).json({
    success: true,
    token,
    user: {
      _id: user._id,
      name: user.name,
      email: user.email,
      role: user.role,
      title: user.title,
      avatar: user.avatar,
      organization: user.organization,
      preferences: user.preferences,
    },
  });
};

// @desc    Register new user & create initial organization
// @route   POST /api/auth/register
const register = async (req, res, next) => {
  try {
    const { name, email, password, organizationName, role } = req.body;

    if (!name || !email || !password) {
      return res.status(400).json({ success: false, message: 'Please provide name, email, and password' });
    }

    const existingUser = await User.findOne({ email: email.toLowerCase() });
    if (existingUser) {
      return res.status(400).json({ success: false, message: 'Email already registered' });
    }

    // Create or find Organization
    const orgName = organizationName || `${name.split(' ')[0]}'s Talent Team`;
    const orgSlug = orgName.toLowerCase().replace(/[^a-z0-9]/g, '-') + '-' + Math.floor(Math.random() * 1000);
    const organization = await Organization.create({
      name: orgName,
      slug: orgSlug,
```

```
63 |     });
64 |
65 |     // Create default scoring config for organization
66 |     await ScoringConfig.create({
67 |       organization: organization._id,
68 |       weights: { skills: 40, semantic: 30, experience: 20, qualification: 10 },
69 |       minimumReviewThreshold: 60,
70 |       shortlistThreshold: 75,
71 |     });
72 |
73 |     // Create user
74 |     const user = await User.create({
75 |       name,
76 |       email: email.toLowerCase(),
77 |       password,
78 |       role: role || 'admin',
79 |       organization: organization._id,
80 |       title: 'Principal Talent Acquisition Lead',
81 |     });
82 |
83 |     await logActivity({
84 |       organizationId: organization._id,
85 |       userId: user._id,
86 |       action: 'USER_REGISTERED',
87 |       entityType: 'Settings',
88 |       entityId: user._id,
89 |       details: { email: user.email, orgName },
90 |       req,
91 |     });
92 |
93 |     sendTokenResponse(user, 201, res);
94 |   } catch (error) {
95 |     next(error);
96 |   }
97 | };
98 |
99 | // @desc    Login user
100 | // @route   POST /api/auth/login
101 | const login = async (req, res, next) => {
102 |   try {
103 |     const { email, password } = req.body;
104 |
105 |     if (!email || !password) {
106 |       return res.status(400).json({ success: false, message: 'Please provide email and p
    > assword' });
107 |     }
108 |
109 |     const user = await User.findOne({ email: email.toLowerCase() }).populate('organizati
    > on');
110 |     if (!user) {
111 |       return res.status(401).json({ success: false, message: 'Invalid credentials' });
112 |     }
113 |
114 |     const isMatch = await user.matchPassword(password);
115 |     if (!isMatch) {
116 |       return res.status(401).json({ success: false, message: 'Invalid credentials' });
117 |     }
118 |
119 |     await logActivity({
120 |       organizationId: user.organization._id,
121 |       userId: user._id,
122 |       action: 'USER_LOGIN',
123 |       entityType: 'Settings',
124 |       entityId: user._id,
125 |       details: { email: user.email },
126 |       req,
127 |     });
128 |
129 |     sendTokenResponse(user, 200, res);
130 |   } catch (error) {
131 |     next(error);
132 |   }
```

```
133 |  };
134 |
135 |  // @desc    Get current user profile
136 |  // @route   GET /api/auth/me
137 |  const getMe = async (req, res, next) => {
138 |    try {
139 |      const user = await User.findById(req.user._id).populate('organization');
140 |      res.status(200).json({
141 |        success: true,
142 |        data: user,
143 |      });
144 |    } catch (error) {
145 |      next(error);
146 |    }
147 |  };
148 |
149 |  // @desc    Logout user & clear cookie
150 |  // @route   POST /api/auth/logout
151 |  const logout = async (req, res) => {
152 |    res.cookie('token', 'none', {
153 |      expires: new Date(Date.now() + 5 * 1000),
154 |      httpOnly: true,
155 |    });
156 |
157 |    res.status(200).json({
158 |      success: true,
159 |      message: 'User logged out successfully',
160 |    });
161 |  };
162 |
163 |  // @desc    Update user profile
164 |  // @route   PUT /api/auth/profile
165 |  const updateProfile = async (req, res, next) => {
166 |    try {
167 |      const { name, title, preferences } = req.body;
168 |      const user = await User.findById(req.user._id);
169 |
170 |      if (name) user.name = name;
171 |      if (title) user.title = title;
172 |      if (preferences) user.preferences = { ...user.preferences, ...preferences };
173 |
174 |      await user.save();
175 |
176 |      res.status(200).json({
177 |        success: true,
178 |        data: user,
179 |      });
180 |    } catch (error) {
181 |      next(error);
182 |    }
183 |  };
184 |
185 |  module.exports = {
186 |    register,
187 |    login,
188 |    getMe,
189 |    logout,
190 |    updateProfile,
191 |  };
192 |
```